Application Development

Lecture 4: Principles of Object-Oriented Programming

Konstantin Kapinchev
University of Greenwich

Application Development
Lecture 4: Principles of Object-Oriented Programming

In this lecture:

- Overview
- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Conclusions

Konstantin Kapinchev

Application Development
Lecture 4: Principles of Object-Oriented Programming

Overview

- **Object-Oriented Programming provides a programming paradigm, which has the ability to model real-world items in a consistent way**

- **This programming paradigm achieves this by applying the following key principles:**
  - **Encapsulation**
  - **Abstraction**
  - **Inheritance**
  - **Polymorphism**

Konstantin Kapinchev

Application Development
Lecture 4: Principles of Object-Oriented Programming

Encapsulation

- **Encapsulation is defined as wrapping data and functionality into a single entity, a class**

- **This wrapping limits the access to data (variables) and functionality (methods) from outside the class**

Encapsulation *continued*

- This access can be limited by utilising the access modifiers
- Most commonly, this is achieved by using:
  - private: in this case, code[1] is accessible only from the same class
  - protected: in this case, code is accessible from the same class and from classes which derived from it

[1]Variables and methods

Encapsulation *continued*

- **Some of the benefits of applying encapsulation include:**
  - **Prevent unwanted or unnecessary access to data and functionality**
  - **Ensure consistency and correctness of the data**
  - **Independence from environment (global data)**
  - **Hide complexity**
  - **And others …**

Encapsulation *continued*

- One way to achieve encapsulation is by using the C# concept of Properties
- They combine features from both variables and methods
- To the user of the class, a property can be considered as variable (data)
- To the developer of the class, a property is implemented by two methods, "get "and "set"
- Example on the next slide illustrates the use of properties

```
class MonthName
{
    // Private data
    private int month = 1;

    //Can be updated only with values from 1 to 12
    public int Month
    {
        get { return (month); }
        set { if ((value > 0) && (value < 13)) { month = value; } }
    }
}

public class MyClass
{
    public static void Main(string[] args)
    {
        String S;
        System.Globalization.DateTimeFormatInfo Months = new System.Globalization.DateTimeFormatInfo();
        MonthName a = new MonthName();

        a.Month = 3;

        S = Convert.ToString(Months.GetMonthName(a.Month));

        Console.WriteLine("The month is: {0}", S);
    }
}
```

The "set" method guarantees month will have value between 1 and 12

Encapsulation implemented by utilising the concept of Property

Application Development
Lecture 4: Principles of Object-Oriented Programming

Abstraction

- During the development of a class, the developer can specify which data and functionality will be accessible, or visible, outside the class and which will not be accessible, or hidden
- This can be done by using the Access Modifiers, the most common of which are "public", "private" and "protected"
- Restricting the access to data and functionality from outside the class is denoted as Abstraction[1] in OOP

[1]Not to be confused with the keyword Abstract, which is about implementing abstraction via incomplete implementation

Konstantin Kapinchev

Abstraction *continued*

- Let's consider the following example:
  - Class "Quadratic Equation", which calculates the solutions of a quadratic equation based on equation coefficients
  - The requirement states that the result needs to be made available via two public variables, namely x1 and x2[1]
  - The source code fragment is on the next slide

[1]Might not be the most optimal implementation, but illustrates the concept of Abstraction

```
class QuadraticEquation
{
    public double a;
    public double b;
    public double c;
    public double D;
    public double? x1;
    public double? x2;

    public void Calculate(double p1, double p2, double p3)
    {
        a = p1;
        b = p2;
        c = p3;
        D = Convert.ToDouble(b * b - 4 * a * c);
        if ((a != 0) && (D >= 0))
        {
            x1 = Convert.ToDouble((-b + Math.Sqrt(D)) / (2 * a));
            x2 = Convert.ToDouble((-b - Math.Sqrt(D)) / (2 * a));
        }
        else
        {
            x1 = null;
            x2 = null;
        }
    }
}
```

Unless it is required from the class, there is no need for these variables to be public

```
class QuadraticEquation
{
    private double a;
    private double b;
    private double c;
    private double D;
    public double? x1;
    public double? x2;

    public void Calculate(double p1, double p2, double p3)
    {
        a = p1;
        b = p2;
        c = p3;
        D = Convert.ToDouble(b * b - 4 * a * c);
        if ((a != 0) && (D >= 0))
        {
            x1 = Convert.ToDouble((-b + Math.Sqrt(D)) / (2 * a));
            x2 = Convert.ToDouble((-b - Math.Sqrt(D)) / (2 * a));
        }
        else
        {
            x1 = null;
            x2 = null;
        }
    }
}
```

These variables can be "abstracted out", or made inaccessible from outside the class

Application Development
Lecture 4: Principles of Object-Oriented Programming

Inheritance[1]

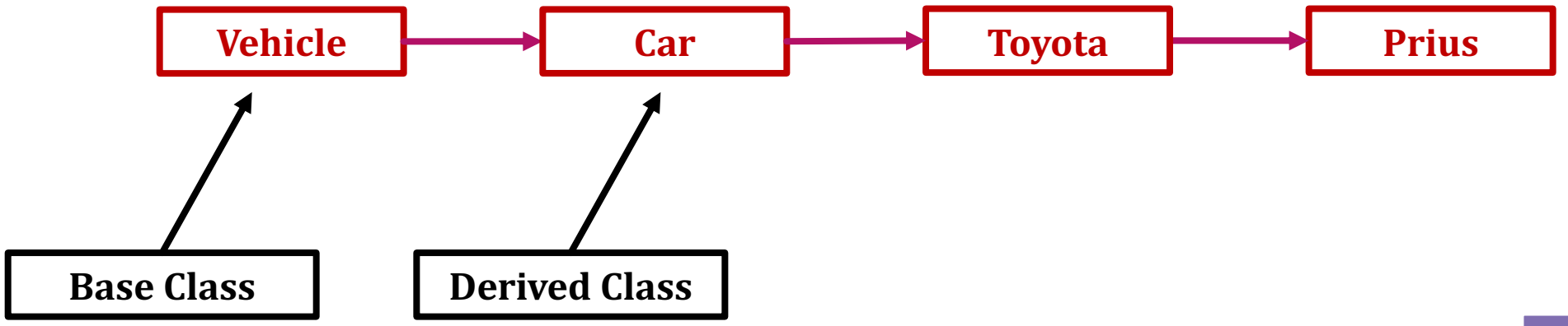
- Inheritance allows us to implement generalisation and specialisation in our programs
- With Inheritance, we can establish connection between more general items and more specific items
- Inheritance can be considered as "is a" relation
- For example:
  - Toyota Pius is a model of Toyota
  - Toyota is a car
  - Car is a motor vehicle

[1]Adapted from "Starting Out with Visual C#", Tony Gaddis, page 623

Application Development
Lecture 4: Principles of Object-Oriented Programming

Inheritance *continued*

- **In Object-Oriented Programming, the more general item is implemented as base class and the more specific item as a derived class**



Inheritance *continued*

- **Example:**
  - **The following example implements classes Vehicle, Car, Toyota and Prius, as described on the previous slides**
  - **Each class has one default constructor and one public method**

```
class Vehicle
{
    public Vehicle(){ Console.WriteLine("This is a vehicle" );}
    public void Say() { Console.WriteLine("Vehicle" ); }
}
class Car : Vehicle
{
    public Car() { Console.WriteLine("This is a car" ); }
    public new void Say() { Console.WriteLine("Car" ); }
}
class Toyota : Car
{
    public Toyota() { Console.WriteLine("This is Toyota"); }
    public new void Say() { Console.WriteLine("Toyota"); }
}
class Prius : Toyota
{
    public Prius() { Console.WriteLine("This is Prius"); }
    public new void Say() { Console.WriteLine("Prius"); }
}
```

By adding ":" followed by a name of a class we specify that class Vehicle will be inherited by class Car. As a result, class Vehicle will be base class and the class Car will be the derived class

Inheritance *continued*

- **Creating an object "a" from class "Prius"**

```
Prius a = new Prius();
```

- **will generate the following output:**

This is a vehicle
This is a car
This is Toyota
This is Prius

Inheritance *continued*

- **Creating an object "a" from class "Prius"**

```
Prius a = new Prius();
```

- **will generate the following output:**

This is a vehicle
This is a car
This is Toyota
This is Prius

Creating a derived class calls the constructor of that class plus the constructor of its base class. This process will repeat for all classes connected with Inheritance

Application Development
Lecture 4: Principles of Object-Oriented Programming

Polymorphism

- **Polymorphism[1] allows multiple methods[2] to share the same name but provide different functionality**
- **The parameters of the methods are expected to be different, which allows the correct method to be called**
- **As a result, different implementations can share the same interface**

[1]Many forms
[2]Those methods can be from different classes

Polymorphism *continued*

- Let's consider the following example, which illustrates static polymorphism[1]
- In this example, multiple methods share the same name
- The correct method is called based on the provided parameters
- Source code fragment on the next slide

[1]Terminology vary across different sources

```
class MyClass
{
    public void Call(int p)
    {
        Console.WriteLine("This is an integer");
    }
    public void Call(double p)
    {
        Console.WriteLine("This is a real number");
    }
    public void Call(char p)
    {
        Console.WriteLine("This is a letter");
    }
    public void Call(string p)
    {
        Console.WriteLine("This is a text");
    }
}
```

Multiple methods share the same name

Note all methods have different parameters

Polymorphism *continued*

- Let's modify the previous example:
  - Method "Say" is now in different classes
  - The base class does not provide implementation for this method[1]
  - In this case, the method "Say" is denoted as abstract[2]
  - All derived methods must implement the method "Say", otherwise the program will generate an error
  - As a result, all derived classes have to implement their own versions of the method "Say"

[1]The base class will provide only the signature of the method
[2]As a result, the class itself will be denoted as abstract too

```
abstract class BaseClass
{
    public abstract void Say();
}

class DerivedClassA : BaseClass
{
    public override void Say() { Console.WriteLine("Drived Class A"); }
}

class DerivedClassB : BaseClass
{
    public override void Say() { Console.WriteLine("Drived Class B"); }
}
```

Note the keywords "abstract" in the base class and "override" in the derived class

```
// BaseClass a = new BaseClass();
DerivedClassA b = new DerivedClassA();
DerivedClassB c = new DerivedClassB();

// a.Say();
b.Say();
c.Say();
```

In this case, an object based on BaseClass can no longer be created

Polymorphism *continued*

- There are other approaches to polymorphism
- Terminology across different sources vary
- For more information and examples:

  Matt Weisfeld, "The Object-Oriented Thought process", page 25

Conclusions

- This lecture took a closer look at the main principles of the object-Oriented Programming, namely Encapsulation, Abstraction, Inheritance and Polymorphism
- These principles were illustrated with examples in C#
- The next lecture will continue studying these principles and will apply some advanced techniques for developing Object-Oriented Solutions

Conclusions *continued*

- **Further reading:**
  - **Matt Weisfeld, "The Object-Oriented Thought process"**
  - **https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/**
  - **https://www.w3schools.com/cs/index.php**